

Report Task 2

Artificial Intelligence and Expert Systems

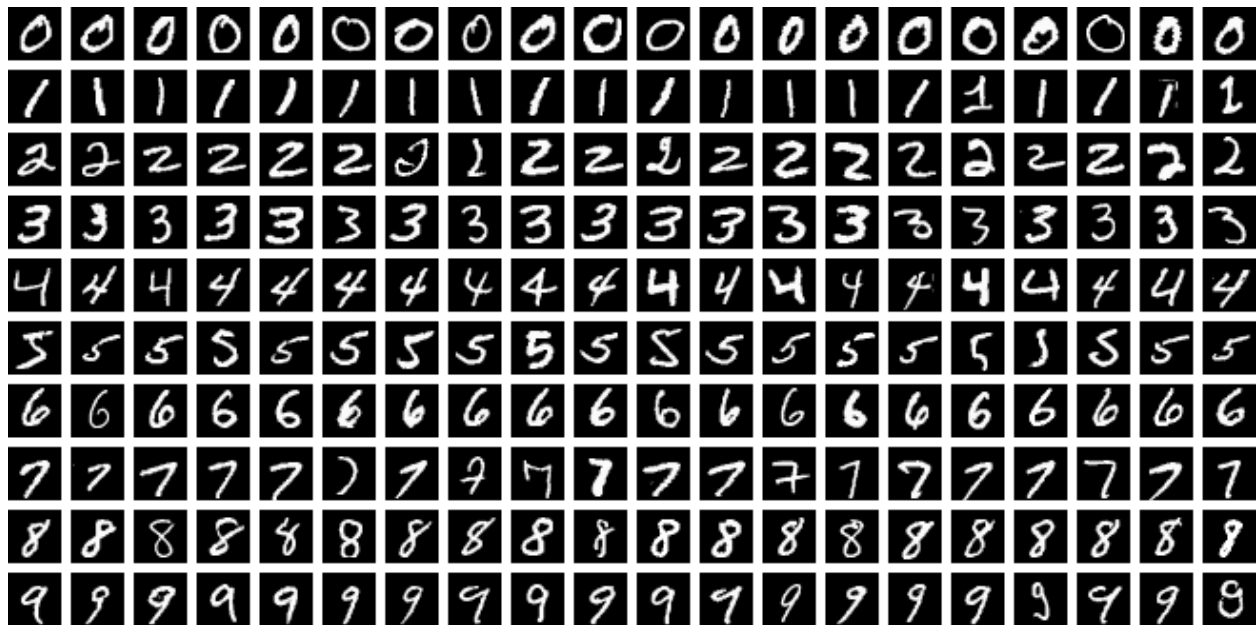
Stanisław Marciniak 247032

Krzysztof Muszyński 247035

1. Dataset

MNIST (Modified National Institute of Standards and Technology database) dataset was used for this project. It consists of 70,000 grayscale images of handwritten digits (0-9), each of size 28x28 pixels. The dataset was split into 60,000 training images and 10,000 test images.

Dataset link: <https://www.kaggle.com/datasets/hojjatk/mnist-dataset>



2. Model

Architecture

A neural network was implemented using TensorFlow/Keras Sequential API. The architecture consists of:

- Input layer:

A *Flatten* layer converts each input image of size **28×28 pixels** into a one-dimensional vector of **784 elements**. This step is necessary because dense (fully connected) layers in Keras require a flat vector as input rather than a 2D image.

- Hidden layers:

The model includes **five Dense layers** with **128 neurons each**, using the **ReLU (Rectified Linear Unit)** activation function. This configuration allows the network to learn complex nonlinear relationships while maintaining efficient gradient propagation. The choice of multiple layers increases the model's capacity to capture hierarchical patterns in the handwritten digits.

- Dropout layer:

A dropout layer was added to prevent overfitting by randomly deactivating 50% of neurons during training. This encourages the model to generalize better and reduces its dependency on specific neurons.

- Output layer:

The final *Dense* layer contains **10 neurons**, each corresponding to one possible digit (0-9). A **softmax activation** function converts the outputs into probabilities (all 10 values sums to 1), enabling multi-class classification. Softmax activation function:

$$\sigma(z_i) = e^{z_i} / \sum_{j=1}^K e^{z_j}$$
$$\sum_{i=1}^K \sigma(z_i) = 1$$

Where:

- K - number of output classes (10 for MNIST)
- z_i - logit for class i
- $\sigma(z_i)$ - predicted probability for class i

Optimization and Loss Functions

The model optimization was performed using the **Adam (Adaptive Moment Estimation)** optimizer. Adam combines the advantages of two popular methods: *Momentum* and *RMSProp*, by maintaining both an exponentially decaying average of past gradients and their squared values.

For each parameter θ_t at iteration t :

$$m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$$

$$v_t = \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$$

$$\hat{m}_t = m_t / (1 - \beta_1^t)$$

$$\hat{v}_t = v_t / (1 - \beta_2^t)$$

$$\theta_{t+1} = \theta_t - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$$

Where:

- g_t - gradient of the loss with respect to parameters at step t
- β_1, β_2 - exponential decay rates (0.9 and 0.999 were used)

- α - learning rate (1e-3 was used)
- ϵ - small constant for numerical stability (1e-7 was used)

Adam was chosen for its **adaptive learning rate** and **robust convergence** across diverse datasets.

The **loss function** used was **Sparse Categorical Cross-Entropy**, which measures the dissimilarity between the predicted probability distribution $y^\wedge$ and the true class label y (encoded as an integer). It is defined as:

$$L(y, \hat{y}) = - (1 / N) \cdot \sum_{i=1}^N \log(\hat{y}_i, y_i)$$

Where:

- N - number of samples
- $\hat{y}_i$ - predicted probability for the true class of sample i
- y_i - true class of sample i

This formulation is suitable when labels are **integer-encoded**, unlike standard categorical cross-entropy which expects **one-hot encoding**.

Training Procedure

The training process consisted of **50 epochs** and a **batch size of 32**.

In mini-batch gradient descent, the model parameters are updated after processing each mini-batch:

$$\theta_{t+1} = \theta_t - \alpha \cdot (1 / B) \cdot \sum_{i=1}^B \nabla_{\theta} L(y_i, \hat{y}_i)$$

Where:

- B – batch size (32 was used)
- α - learning rate (1e-3 was used)
- $\nabla_{\theta} L(y_i, \hat{y}_i)$ – gradient of the loss for the i -th example

This approach provides a trade-off between **stability** (as in full-batch gradient descent) and **efficiency** (as in stochastic gradient descent).

3. Data preprocessing

The preprocessing pipeline was designed to transform the user-drawn input image into a format consistent with the MNIST dataset used during model training. This ensures that the neural network receives standardized input, which improves both prediction accuracy and model stability.

The process consisted of several stages:

Image Acquisition and Decoding

The digit drawn by the user on the web interface was transmitted as a Base64-encoded image. The server decoded this data into a numerical image representation that could be processed using computer vision methods.

This conversion enables consistent numerical representation of the input, regardless of browser or device differences.

Grayscale Conversion

The decoded RGB image was converted to a single-channel grayscale format.

The MNIST dataset consists of grayscale images; converting the input ensures that the model operates on the same input domain and reduces computational complexity by eliminating unnecessary color channels.

Digit Localization and Cropping

The algorithm detected the region containing the drawn digit and cropped it from the surrounding background.

Removing empty areas ensures that the model focuses on the relevant part of the image, improving recognition and reducing noise.

Resizing and Aspect Ratio Preservation

The cropped digit was resized to fit within a 20×20 pixel area (4px padding), maintaining its aspect ratio.

Consistent scaling allows digits of different sizes to be represented uniformly, while preserving proportions prevents distortion that could negatively affect feature extraction.

Centering within a 28×28 Frame

The resized digit was placed in the center of a 28×28 pixel frame, replicating the format of MNIST images, which include uniform padding around the digit.

Centering and padding standardize the spatial distribution of pixels, which helps the model generalize better by matching the spatial layout of the training data.

Dimensional Adjustment for Model Input

The final image was reshaped to match the expected input dimensions of the neural network, treating the sample as a single-element batch.

Ensuring dimensional compatibility prevents inference errors and aligns the input structure with that used during training.

4. Model's accuracy

The model demonstrated excellent performance during both training and evaluation. Over the course of 50 epochs, training and validation accuracies increased steadily, indicating effective learning and minimal signs of overfitting.

By the final epoch, the model achieved the following results:

- **Training Accuracy:** 99.73%
- **Validation Accuracy:** 97.93%
- **Test Accuracy:** 97.97%
- **Test Loss:** 0.25

These outcomes confirm that the neural network generalizes well to unseen data and maintains stable predictive performance across different datasets.

To assess performance beyond simple accuracy, the model was evaluated using **precision**, **recall**, and the **F1-score**, which provide a more detailed view of classification quality for each digit class:

- **Precision** measures the proportion of correctly predicted samples among all predictions made for a given class, reflecting how *reliable* the model's positive predictions are. A high precision value indicates that the model rarely misclassifies other digits as that class.
- **Recall** (or *sensitivity*) measures the proportion of correctly identified samples among all actual instances of a class, capturing how *completely* the model detects all relevant examples. High recall means that most true instances are successfully recognized.
- **F1-score** represents the harmonic mean of precision and recall, providing a balanced metric that considers both false positives and false negatives. It is particularly valuable in multi-class problems where class distributions and error types may vary.

The model achieved **average precision, recall, and F1-scores of approximately 0.98**, demonstrating consistent and balanced performance across all ten digit classes. Digits such as “1” and “0” were recognized with near-perfect accuracy due to their distinct shapes, while slightly lower recall values were observed for “4”, “5”, and “8”, which are more visually similar and occasionally confused.

Overall, the results indicate that the model performs with high accuracy, precision, and generalization capability, reliably distinguishing handwritten digits across the MNIST dataset.

```

=== Evaluating on Test Data ===
Test Loss: 0.2518
Test Accuracy: 0.9797

=== Final Training Metrics ===
Training Loss: 0.0122
Training Accuracy: 0.9973
Validation Loss: 0.2607
Validation Accuracy: 0.9793

=== Classification Report ===

```

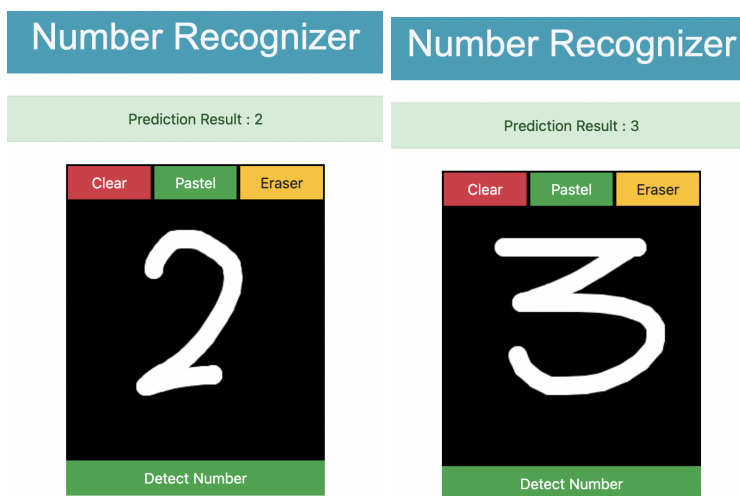
	precision	recall	f1-score	support
0	0.97	0.99	0.98	980
1	0.99	0.99	0.99	1135
2	0.97	0.98	0.98	1032
3	0.97	0.99	0.98	1010
4	0.99	0.97	0.98	982
5	0.99	0.97	0.98	892
6	0.99	0.98	0.98	958
7	0.98	0.98	0.98	1028
8	0.97	0.97	0.97	974
9	0.98	0.98	0.98	1009
accuracy			0.98	10000
macro avg	0.98	0.98	0.98	10000
weighted avg	0.98	0.98	0.98	10000

5. Conclusions

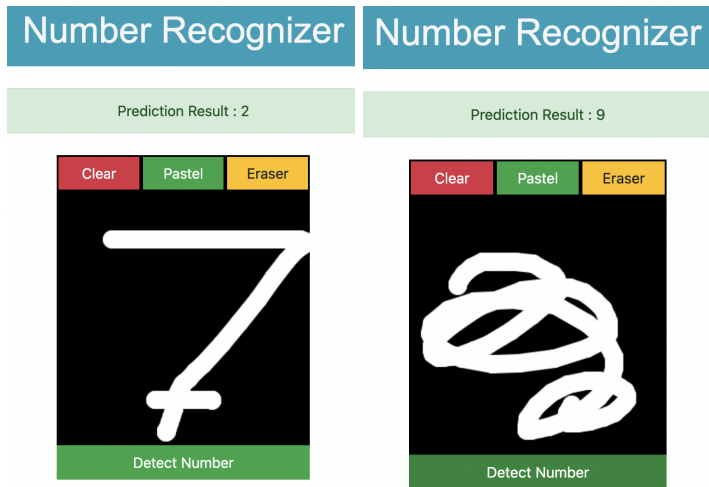
The application functions as intended — users can draw numbers directly on a canvas rendered in the web browser, and the connected model successfully predicts which number has been drawn.

The model's accuracy is strong; predictions are almost always correct when the digits are drawn clearly and neatly.

Examples of correct predictions:



Examples of incorrect predictions:



When the drawing is sloppy or ambiguous, the model's predictions tend to deviate.

Overall, the model demonstrates solid performance and behaves as expected.